

IMAGE PROCESSING AND ITS APPLICATION

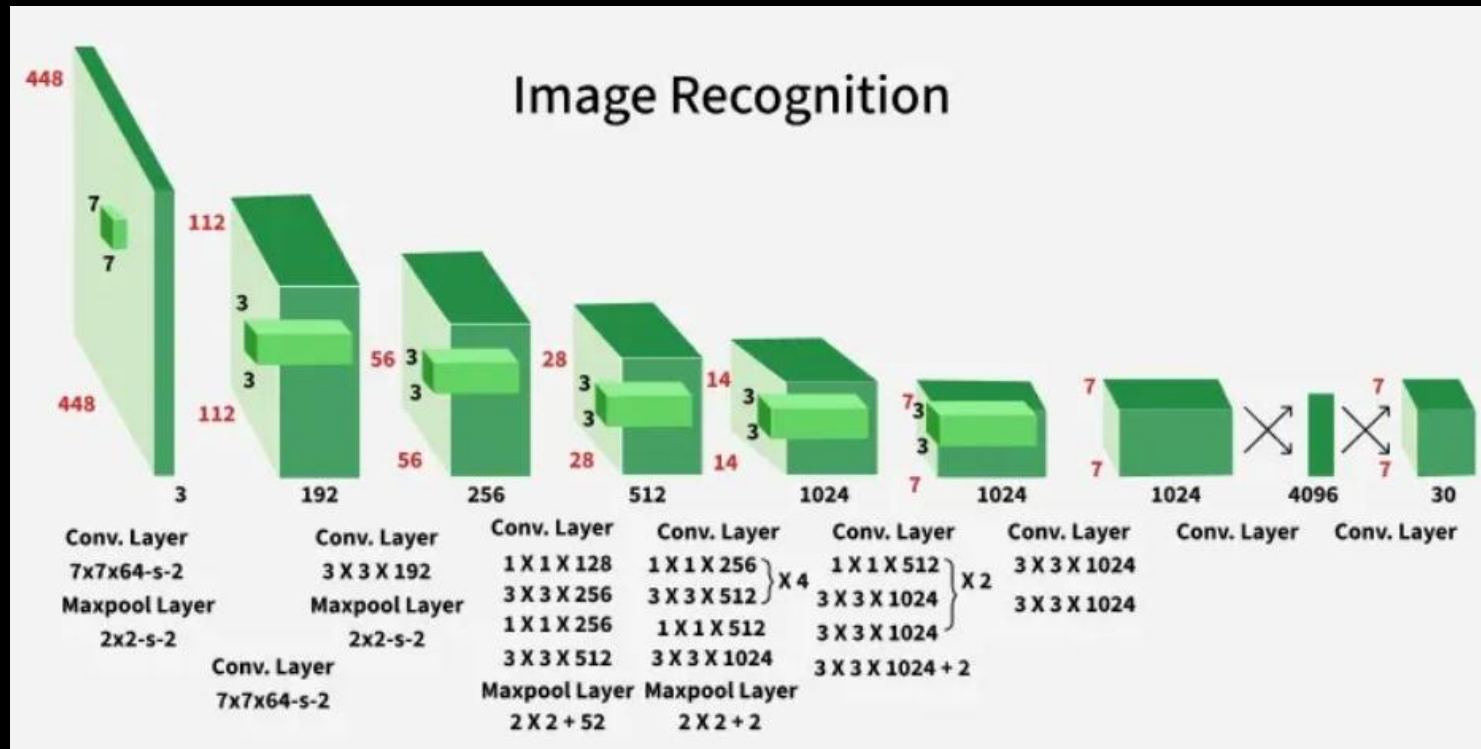
MODULE - 5

Prof. Nishchitha MH
Assistant Professor
Dept. of Robotics and Artificial Intelligence

YOLO Architecture

YOLO is a **one-stage object detection algorithm** that detects objects in a **single pass** through the neural network, making it very fast and suitable for real-time applications.

- Instead of checking many regions separately, YOLO:
- Looks at the **entire image once**
- Divides it into a **grid**
- Predicts **bounding boxes + class probabilities** directly



YOLO Architecture

1. Input Preprocessing:

- The model accepts an image as input.
- It resizes the input image to 448×448 pixels ensuring that the aspect ratio is preserved using padding.
- This ensures uniformity of input dimensions across the network which is essential for batch processing in deep learning.

2. Backbone Convolutional Neural Network (CNN):

- After preprocessing the image is passed through a deep CNN architecture designed for object detection:
- The model consists of **24 convolutional layers** and **4 max-pooling layers**.
- These layers help in extracting hierarchical spatial features from the image.

YOLO Architecture

3. Use of 1×1 and 3×3 Convolutions:

- To reduce the number of parameters and compress channels, 1×1 convolutions are employed.
- These are followed by 3×3 convolutions to capture spatial patterns in the feature maps.
- This design pattern i.e 1×1 followed by 3×3 improves computational efficiency while maintaining expressive power.

4. Fully Connected Layers:

Following the convolutional layers, the architecture has 2 fully connected layers. The final fully connected layer produces an output of shape (1, 1470).

5. Cuboidal Prediction Output:

The output vector of size 1470 is reshaped to (7, 7, 30). Here, 7×7 represents the grid cells, and 30 represents the prediction vector for each cell:

$$30 = (2 \text{ bounding boxes} \times 5) + (20 \text{ class probabilities})$$

YOLO Architecture

6. Activation Functions:

The architecture predominantly uses Leaky ReLU as its activation function.

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ 0.01x, & \text{if } x \leq 0 \end{cases}$$

This activation allows a small gradient when the unit is not active, preventing dead neurons.

7. Output Layer Activation:

The last layer uses a linear activation function, suitable for making raw predictions like bounding box coordinates and confidence scores.

8. Regularization Techniques:

- **Batch Normalization** is employed across layers to stabilize and accelerate training.
- **Dropout** is also incorporated to prevent overfitting by randomly deactivating neurons during training, encouraging the network to learn more robust features.

Working of YOLO

Step 1: Input Image

Input image (e.g., $416 \times 416 \times 3$) is given to the CNN.

Step 2: Divide Image into Grid

The image is divided into an $S \times S$ grid.

Example: 7×7 grid / 9×9

Each grid cell is responsible for detecting objects whose **center** lies inside it.



Working of YOLO

Step 3: Each Grid Cell Predicts Bounding Boxes

Each grid cell predicts:

B bounding boxes

Each box contains:

$x, y \rightarrow$ center of box

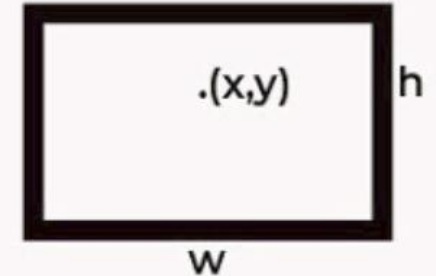
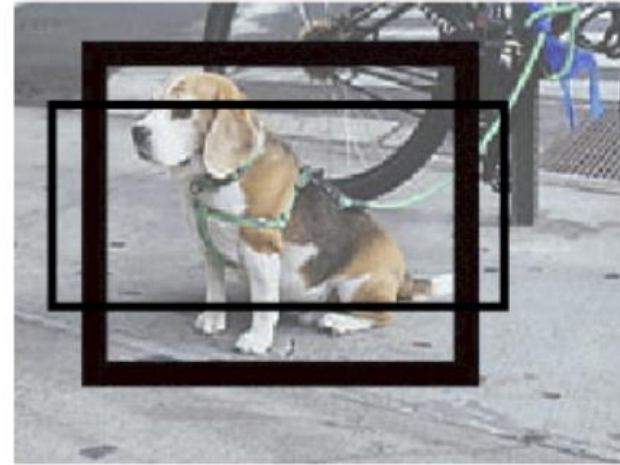
$w, h \rightarrow$ width and height

Confidence score

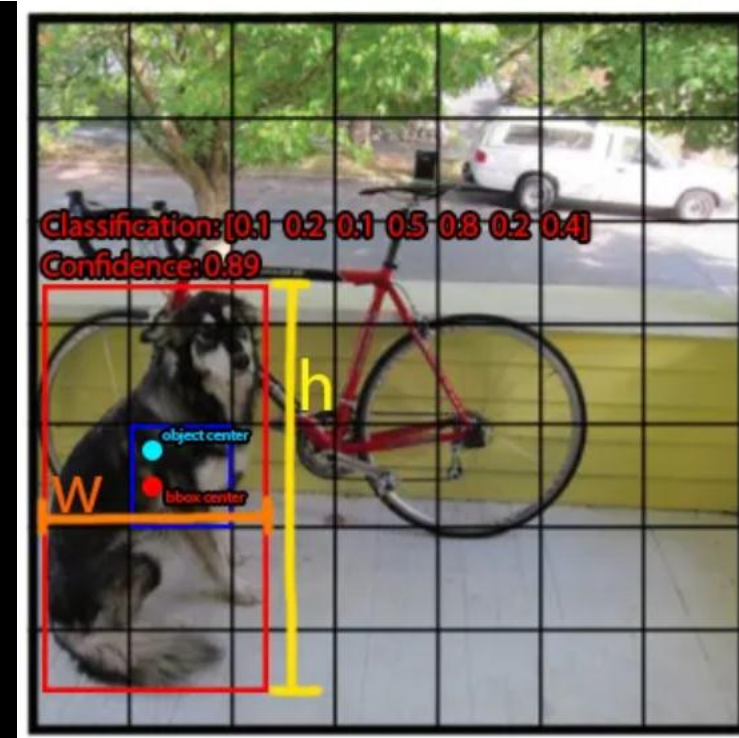
So each box = 5 values:

$(x, y, w, h, confidence)$

Each box predict:



$P(\text{Object})$: Probability that the box contains an object



Working of YOLO

Step 4: Class Probability Prediction

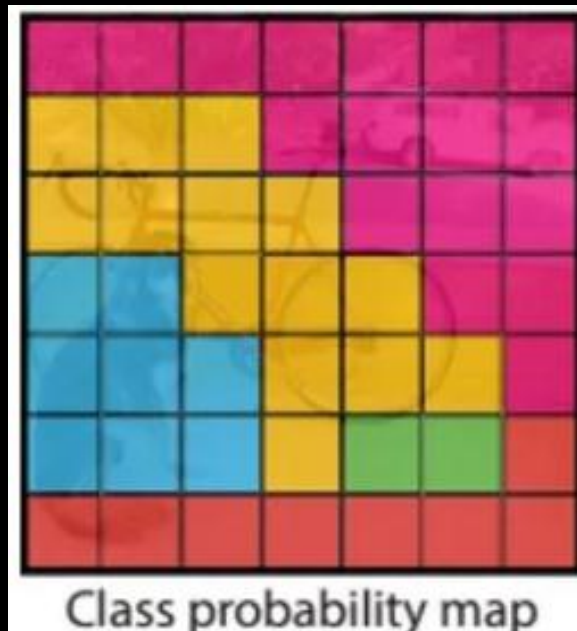
Each grid cell also predicts:
Probability of each class

Example:

Person = 0.8

Dog = 0.1

Car = 0.1



Step 5: Final Prediction per Grid Cell

Each grid cell outputs:

$$B \times 5 + C$$

Where:

B = number of bounding boxes

C = number of classes



Working of YOLO

Step 6: Combine Predictions

The model combines:

- Bounding box predictions
- Class probabilities

To produce: Final detected objects

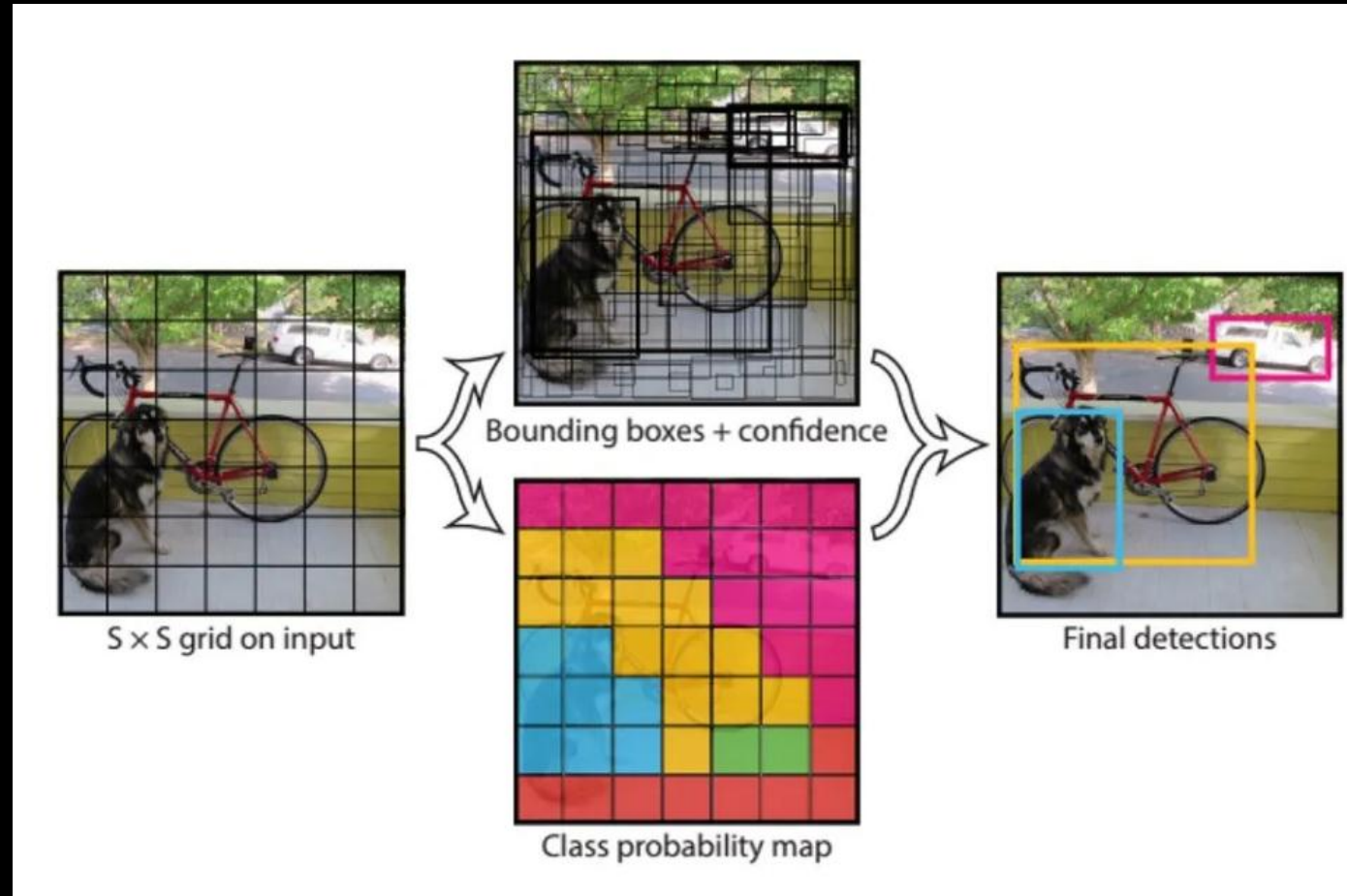
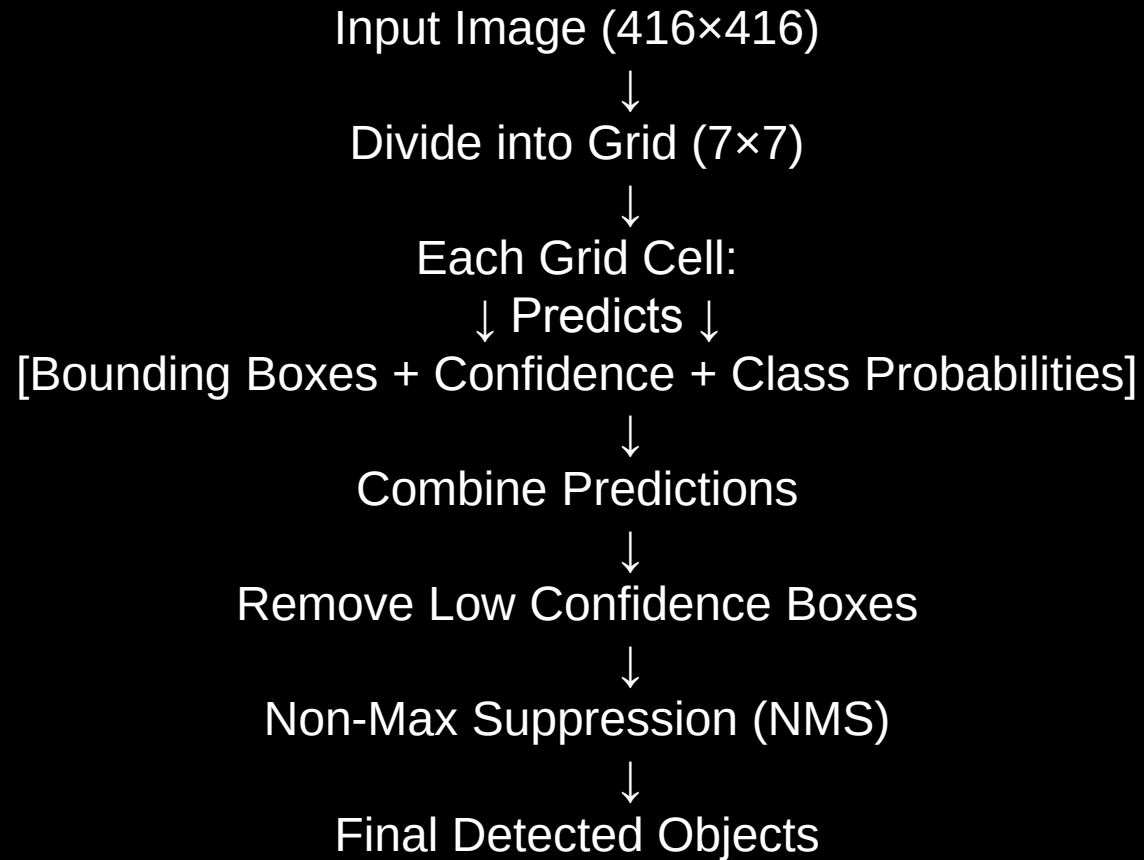
Step 7: Apply Threshold

Remove low-confidence detections:
Keep only boxes with **high confidence score**

Step 8: Non-Max Suppression (NMS)

Removes duplicate boxes:
If multiple boxes detect the same object
Keep only the best one

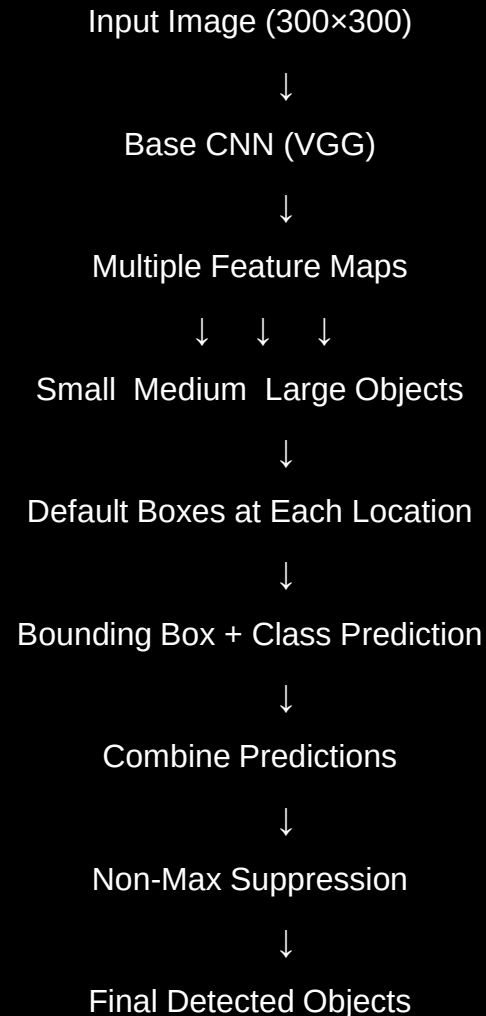
Working of YOLO



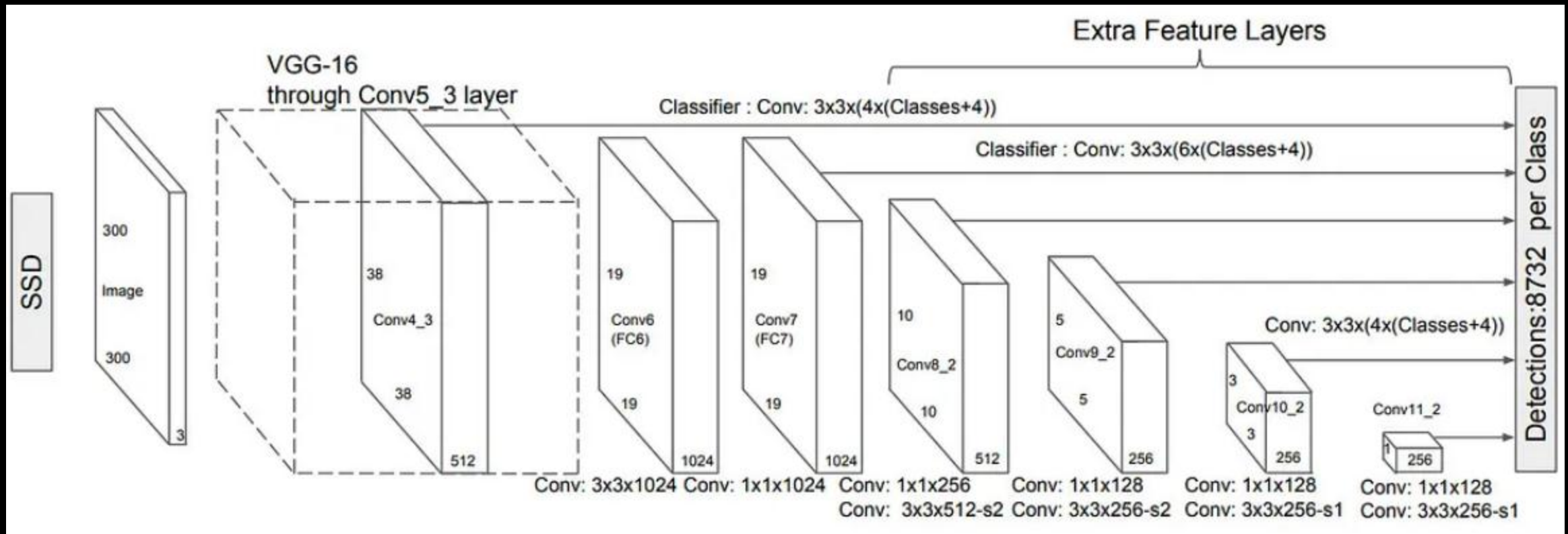
SSD Object Detection – Single Shot MultiBox Detector

- **SSD** is a **one-stage object detection model** that detects objects in a **single forward pass** (single shot), making it **fast and efficient** for real-time applications.
 - **Single Shot** → Detection is done in **one pass** (no region proposal stage)
 - **MultiBox** → Uses **multiple bounding boxes (default boxes)** at different locations and scales
- Uses a **CNN backbone** (like **VGGNet**)
- Extracts features at **multiple scales**
- Predicts:
 - Bounding boxes
 - Class probabilities

SSD Object Detection – Single Shot MultiBox Detector



SSD Object Detection – Single Shot MultiBox Detector



SSD Object Detection – Single Shot MultiBox Detector

Step 1: Input image – 300x300x3

Step 2: Feature Extraction (Base Network)

- Uses pre-trained CNN (e.g., VGG-16)
- Extracts feature maps

SSD Object Detection – Single Shot MultiBox Detector

Step 3: Multi-Scale Feature Maps

SSD uses **multiple feature maps** of different sizes:

Feature Map	Size	Purpose
Conv4_3	Large	Detect small objects
Conv7	Medium	Medium objects
Conv8_2	Smaller	Large objects
Conv9_2	Smaller	Larger objects
Conv10_2	Very small	Very large objects
Conv11_2	Smallest	Biggest objects

SSD Object Detection – Single Shot MultiBox Detector

Step 4: Default Boxes (Anchor Boxes)

At each location in a feature map:

Multiple **default boxes** are assigned

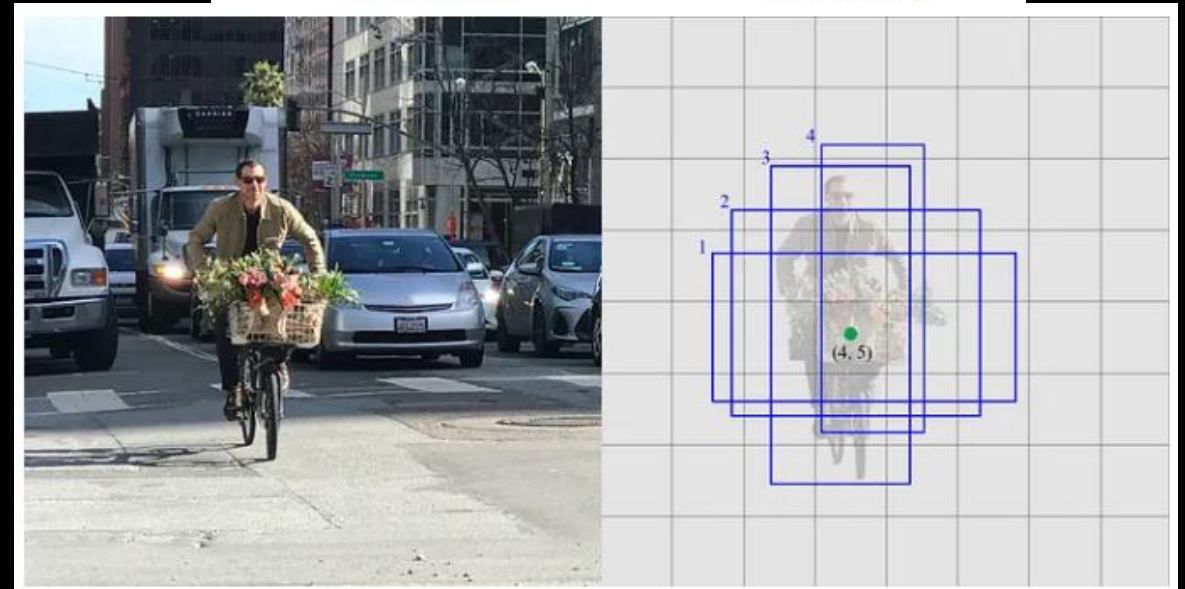
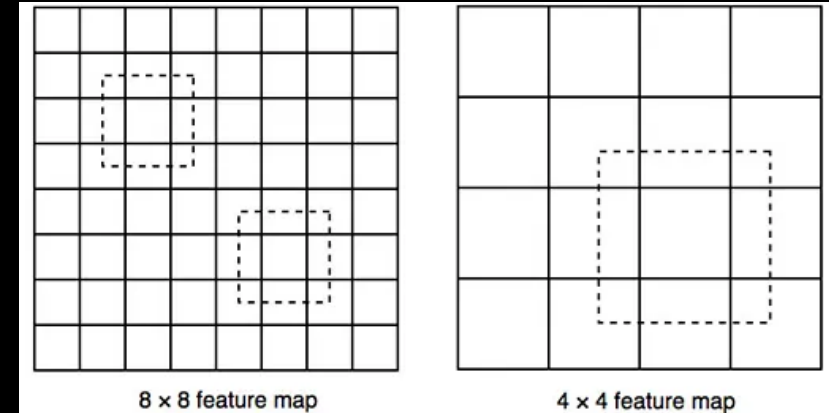
Each box has:

- Different aspect ratios

- Different scales

Example:

1:1, 2:1, 1:2, etc.



SSD Object Detection – Single Shot MultiBox Detector

Step 5: Predictions at Each Location

Each default box predicts:

4 values $\rightarrow$ bounding box (x, y, w, h)

C values $\rightarrow$ class probabilities

Total:

$4+C$

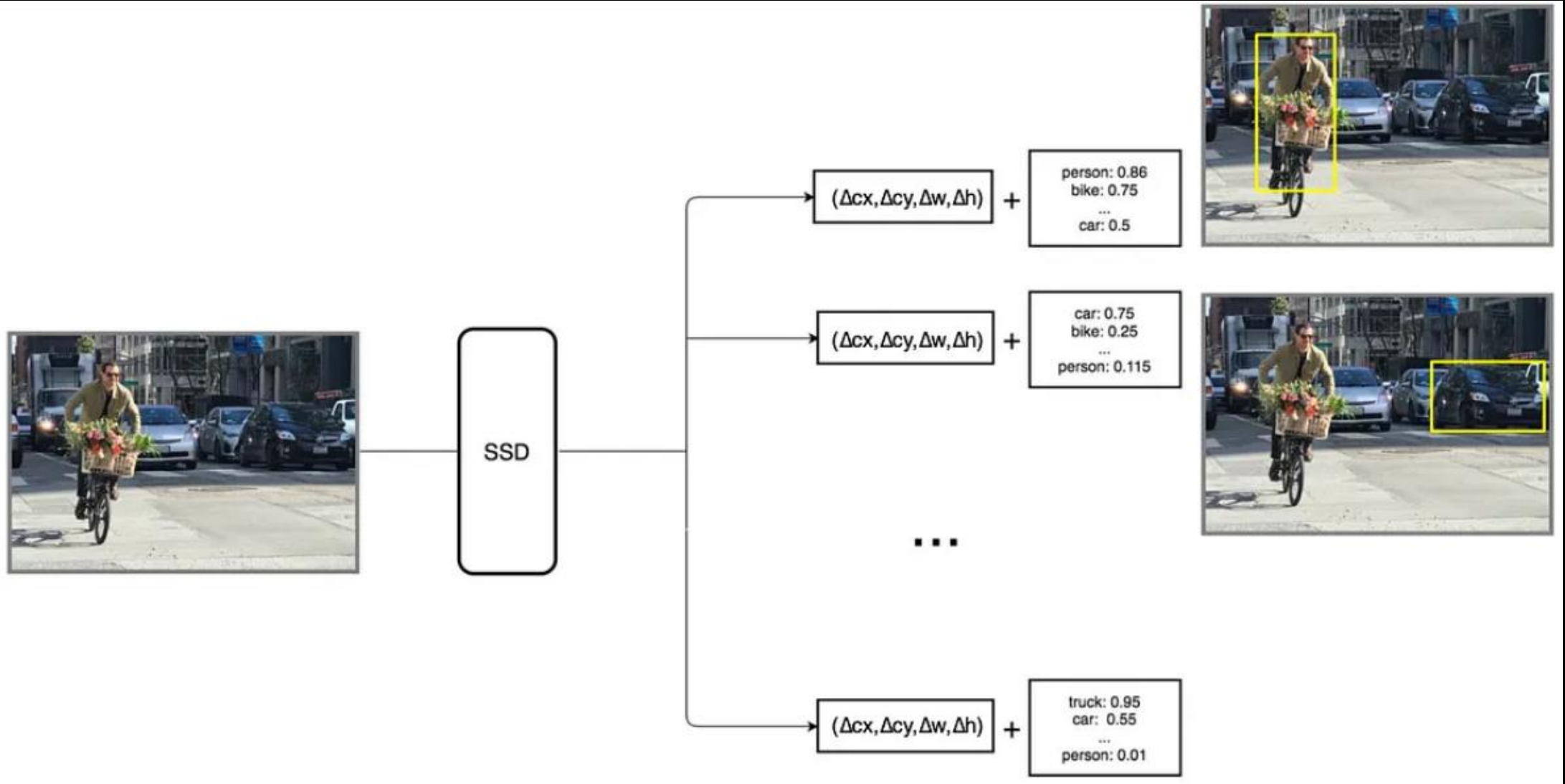
Step 6: Combine All Predictions - All predictions from all feature maps are combined.

Step 7: Apply Threshold - Remove low-confidence boxes

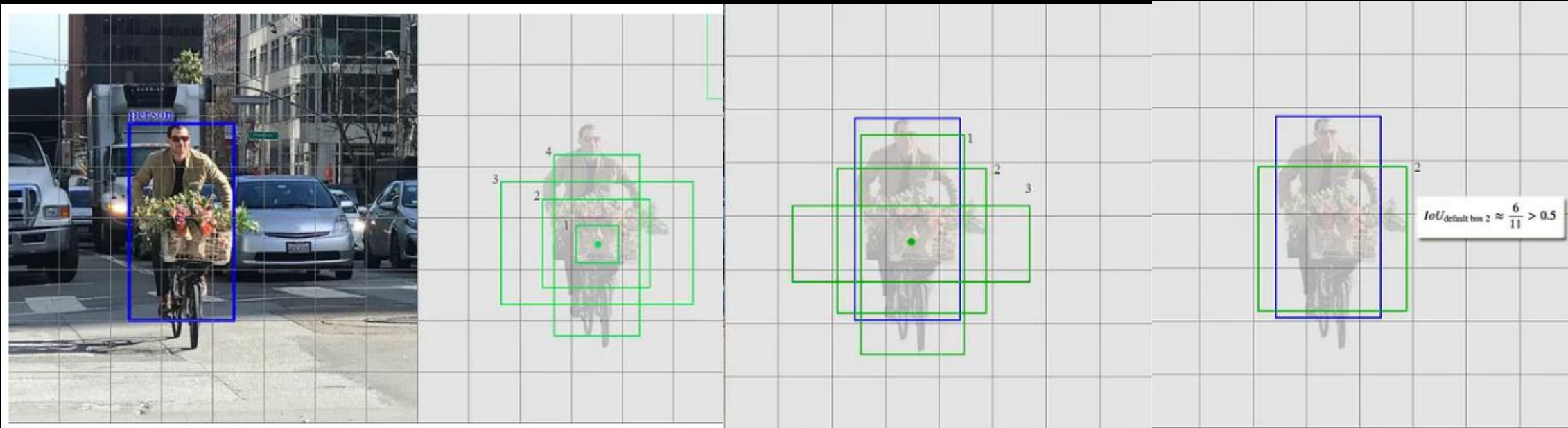
Step 8: Non-Max Suppression (NMS)

- Remove duplicate boxes
- Keep best bounding box

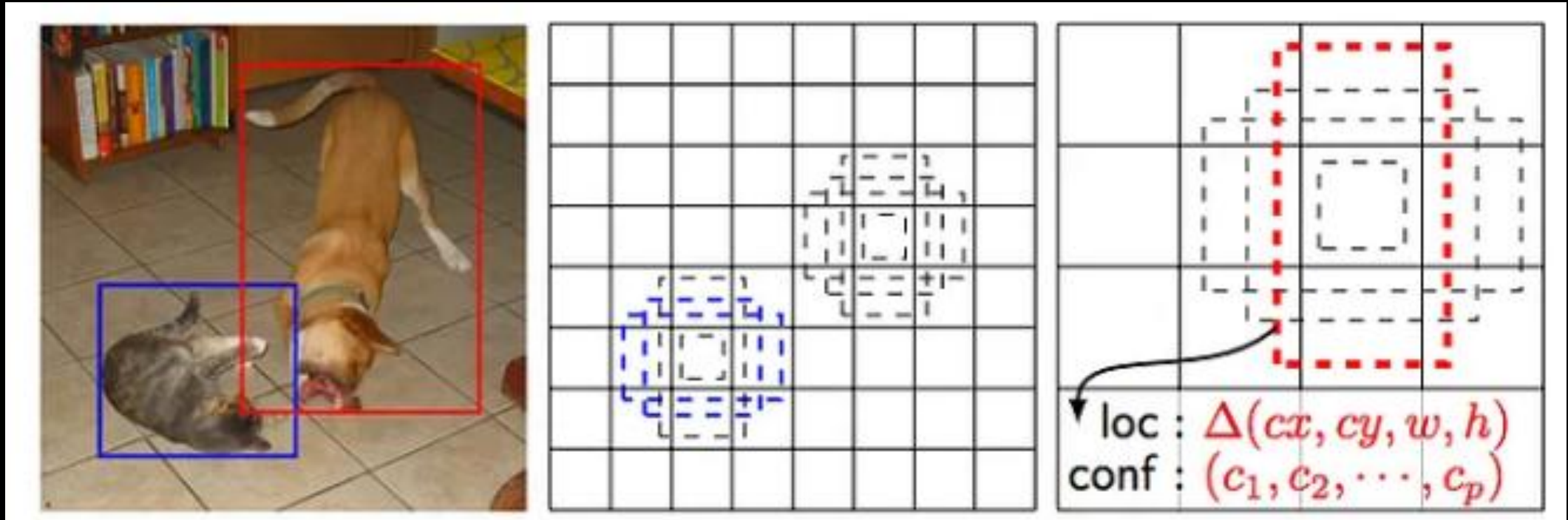
SSD Object Detection – Single Shot MultiBox Detector



SSD Object Detection – Single Shot MultiBox Detector



SSD Object Detection – Single Shot MultiBox Detector



Residual Networks (ResNet)

To overcome the challenges of training very deep neural networks, Residual Networks (ResNet) was introduced, which uses skip connections that allow the model to learn residual mappings instead of direct transformations making deep neural networks easier to train.

- It helps prevent vanishing gradient problems in very deep models.
- Skip connections let information flow directly across layers.
- ResNet enables building networks with hundreds or even thousands of layers.
- It is widely used in computer vision tasks like image classification and object detection.

Residual Networks (ResNet)

Two major issues are:

1. **Vanishing/Exploding Gradient Problem:** As the number of layers in a neural network increases, the gradients of the loss function with respect to the weights can become extremely small or excessively large during backpropagation.
2. **Degradation Problem:** The degradation problem occurs when increasing the network depth does not improve performance and may even worsen it. This problem has two aspects:
 - **Performance Plateau:** Training error saturates after a certain depth meaning additional layers do not significantly reduce the error.
 - **Accuracy Degradation:** Beyond a certain depth validation error increases and the model performs poorly on unseen data.

Residual Networks (ResNet)

Core Idea: Residual Learning

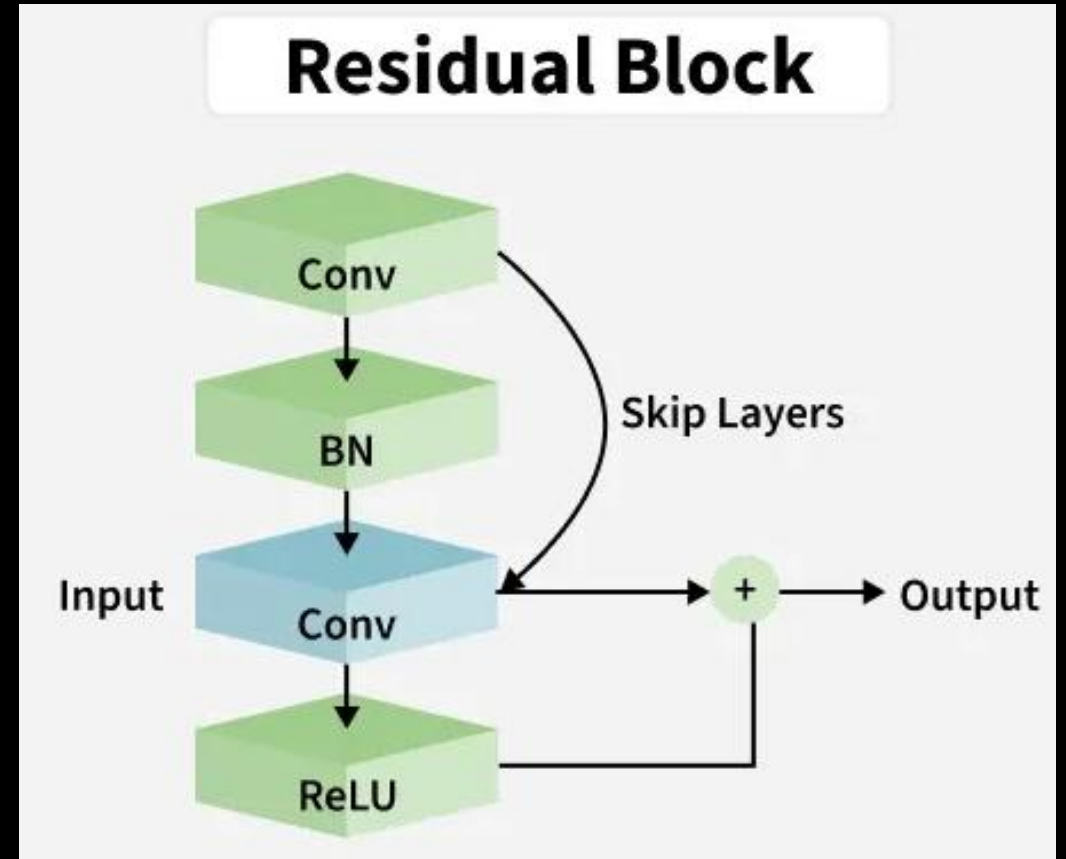
Instead of directly learning: $H(x)$

ResNet learns: $F(x) = H(x) - x$

output becomes: $H(x) = F(x) + x$

This is implemented using a skip connection.

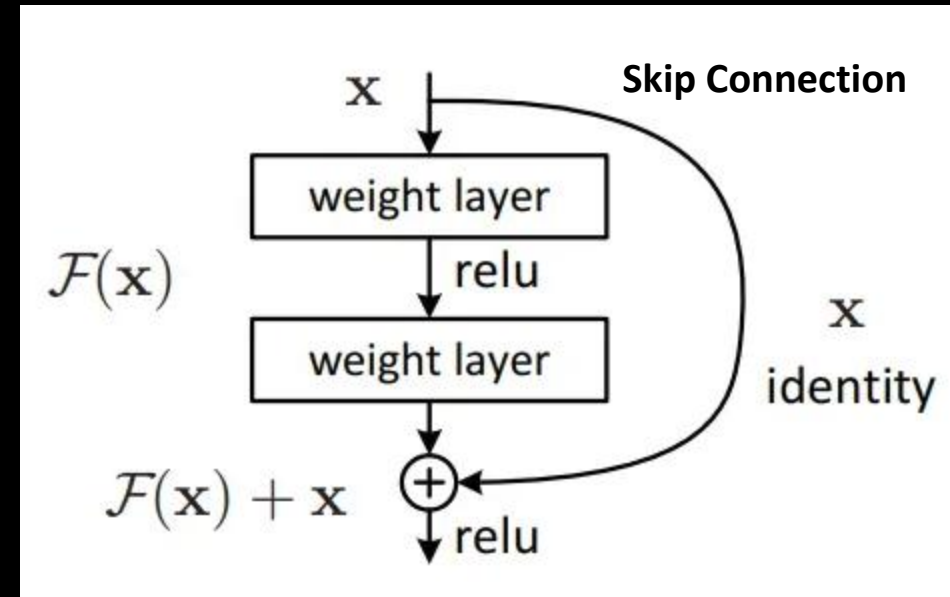
Learning the simpler residual $F(x)$ makes optimization easier.



Residual Networks (ResNet)

1. Residual Block: A Residual Block contains:

- One or more convolutional layers
- A skip connection that bypasses these layers
- Addition of input to convolution output
- This ensures uninterrupted flow of information and gradients



Residual Networks (ResNet)

2. Skip (Shortcut) Connection

- Bypasses one or more layers
- Adds input directly to output
- Prevents vanishing gradients
- Improves parameter updates

3. Handling Dimension Mismatch: When input and output dimensions differ

- **Zero Padding:** Adds extra zeros to the input to match output dimensions in a residual block
- **Linear Projection:** Uses a learnable 1×1 convolution to match input and output dimensions for the skip connection.

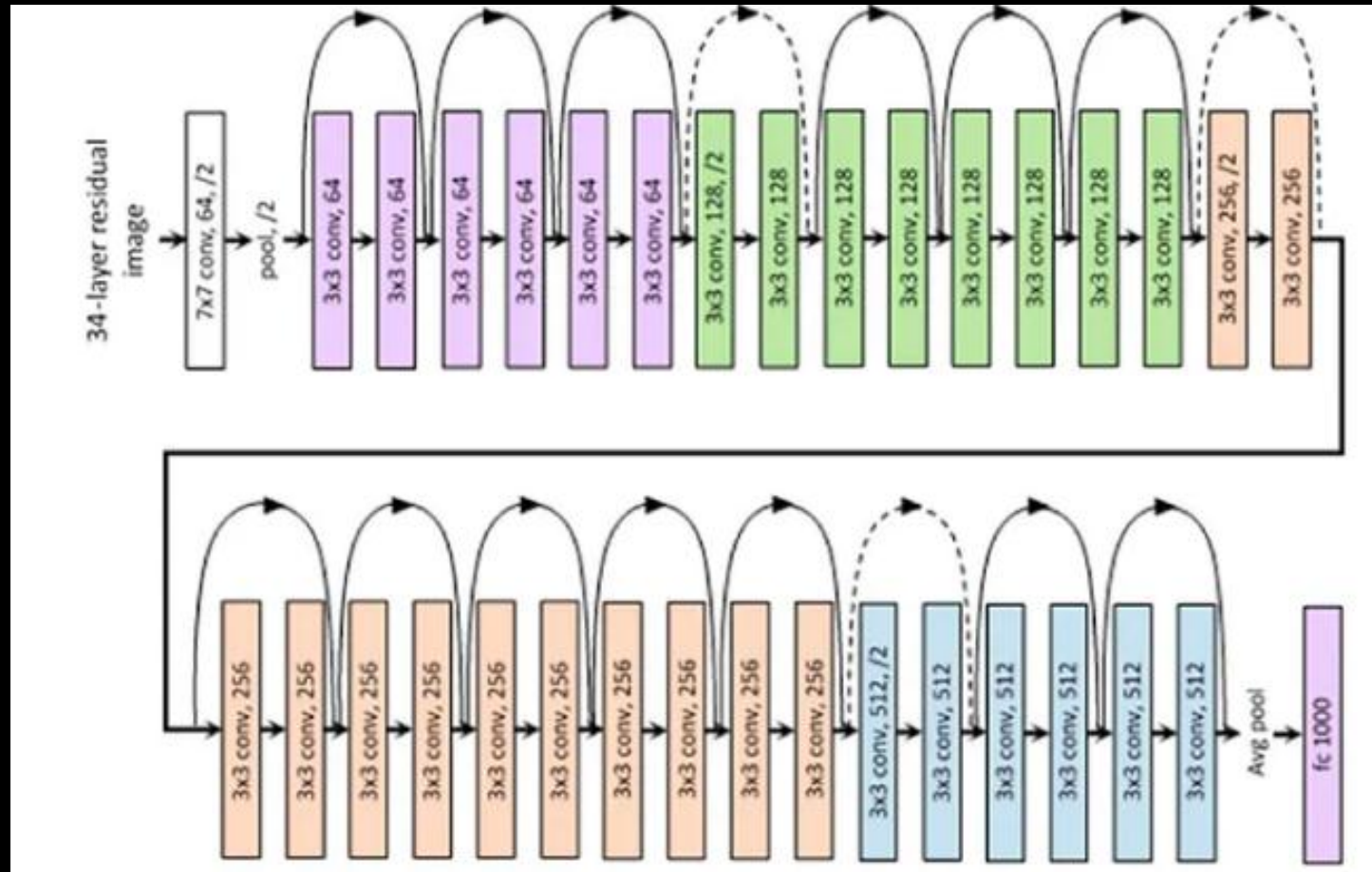
Residual Networks (ResNet)

3. Stacking Residual Blocks : Multiple residual blocks can be stacked to create deep architectures. This allows networks to go very deep without suffering from degradation.

4. Global Average Pooling (GAP): Before the final fully connected layer ResNet uses GAP:

- Converts each feature map to a single value by averaging
- Reduces parameters less overfitting
- Produces compact feature representation

Residual Networks (ResNet)

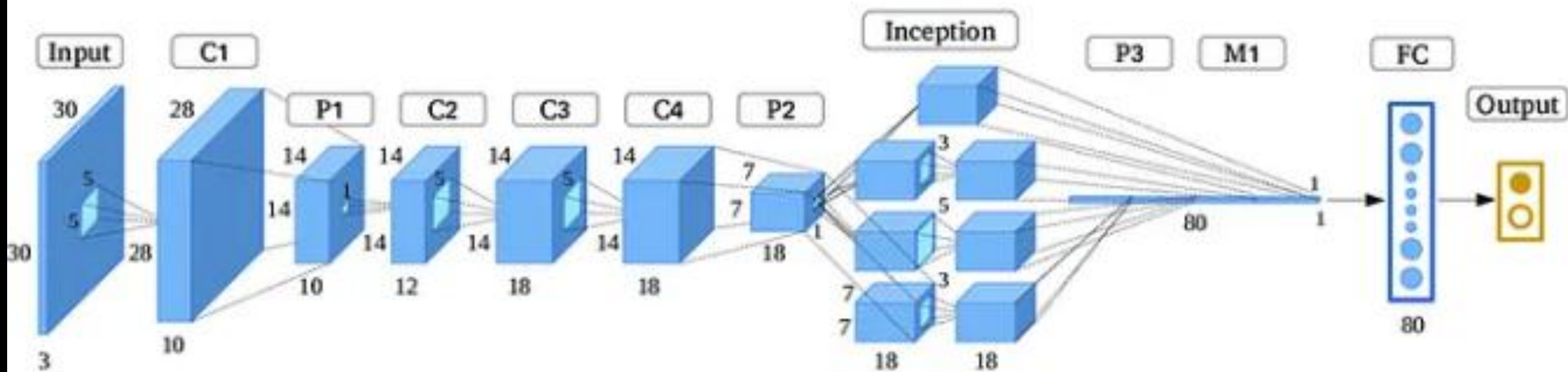


ResNet is a deep convolutional neural network that uses residual (skip) connections to enable efficient training of very deep networks by overcoming the vanishing gradient problem.

GoogLeNet Model

- GoogLeNet (Inception V1) is a deep convolutional neural network architecture designed for efficient image classification.
- It introduces the Inception module, which performs multiple convolution operations (1x1, 3x3, 5x5) in parallel, along with max pooling and concatenates their outputs.
- The architecture is deep, yet optimized for speed and performance, which makes it suitable for large-scale visual recognition tasks.
- It brought forward innovative architectural choices such as 1x1 convolutions, global average pooling and the Inception module, all aimed at improving depth and computational efficiency.

GoogLeNet Model



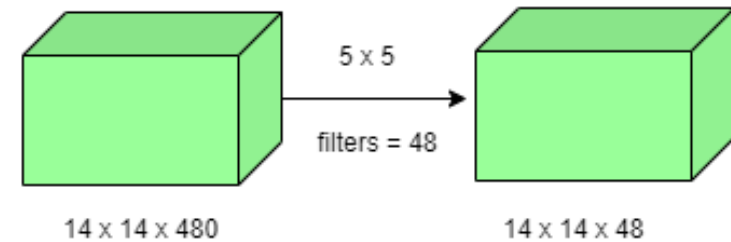
GoogLeNet (Inception v1) architecture

Key Features of GoogLeNet

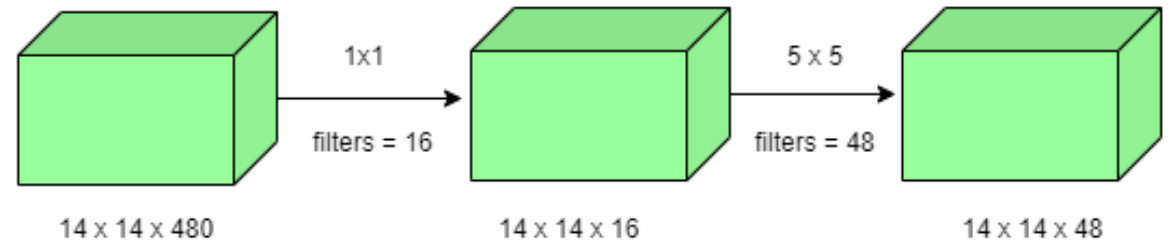
1×1 Convolutions

- One of the core techniques employed in GoogLeNet is the use of 1×1 convolutions, primarily for dimensionality reduction.
- These layers help decrease the number of trainable parameters while enabling deeper and more efficient architectures.
- This results in a massive reduction in computation without compromising performance.

Without 1×1 Convolution: $(14 \times 14 \times 48) \times (5 \times 5 \times 480) = 112.9\text{M}$ operations



With 1×1 Convolution: $(14 \times 14 \times 16) \times (1 \times 1 \times 480) + (14 \times 14 \times 48) \times (5 \times 5 \times 16) = 5.3\text{M}$ operations



GoogLeNet Model

2. Global Average Pooling

In traditional architectures like AlexNet, **fully connected layers** at the end introduce a large number of parameters. GoogLeNet **replaces** these with **Global Average Pooling**, which computes the average of each feature map (e.g. **converting 7×7 maps to 1×1**), this significantly reduces the model's parameter count and solves overfitting.

Benefits:

- Zero additional trainable parameters
- Reduces overfitting
- Improves top-1 accuracy by approximately 0.6%

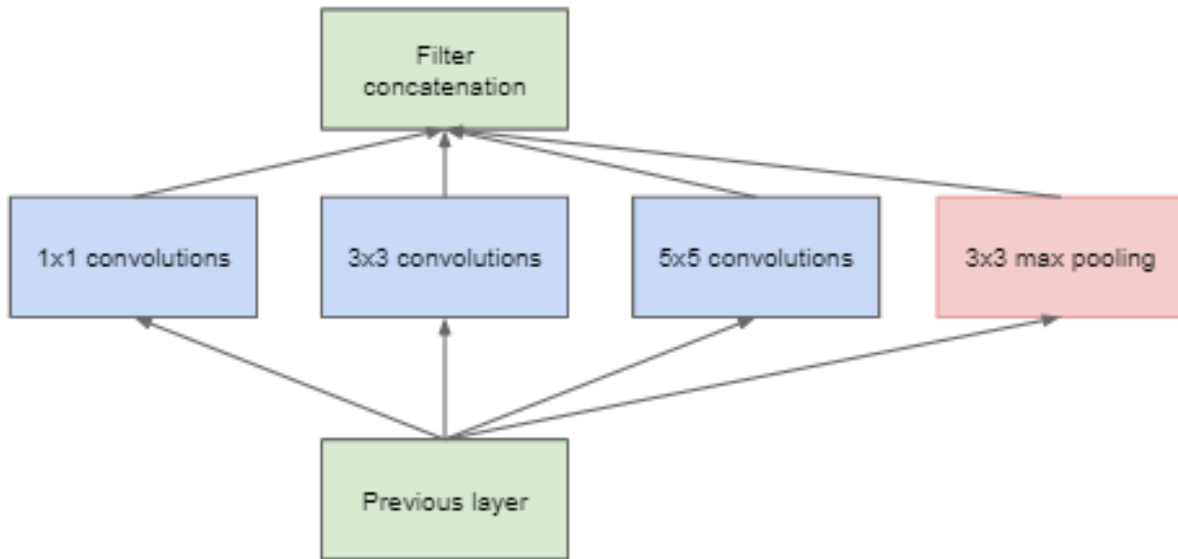
GoogLeNet Model

3. Inception Module

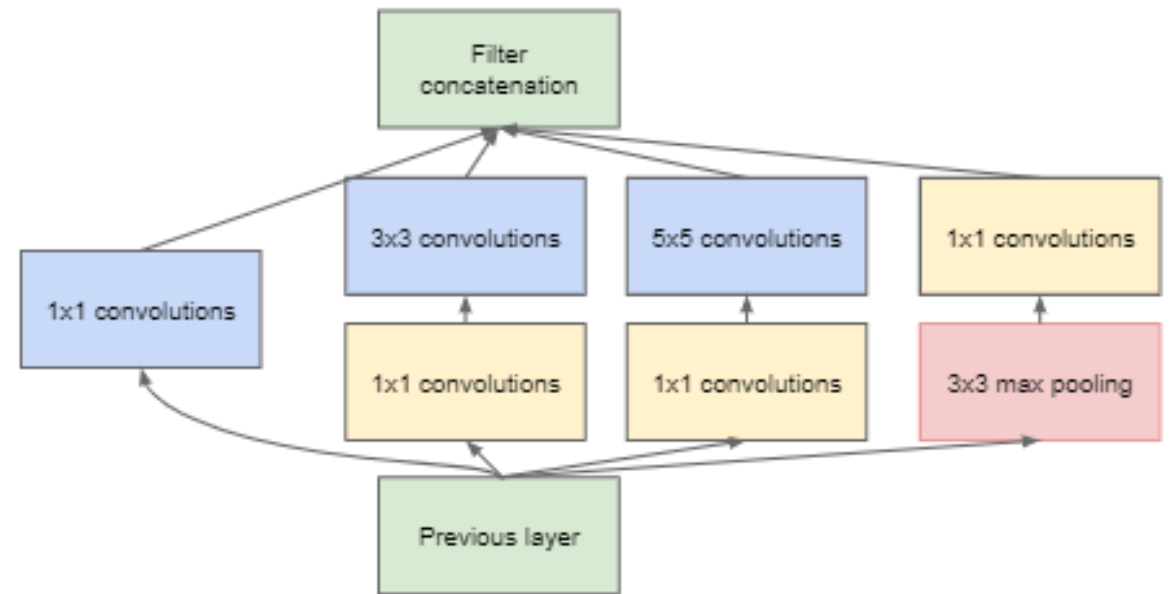
The Inception module is the architectural core of GoogLeNet. It processes the input using multiple types of operations in parallel, including 1×1 , 3×3 , 5×5 convolutions and 3×3 max pooling. The outputs from all paths are concatenated depth-wise.

- **Purpose:** Enables the network to capture features at **multiple scales** effectively.
- **Advantage:** Improves representational power without dramatically increasing computation.

GoogLeNet Model



(a) Inception module, naïve version



(b) Inception module with dimension reductions

GoogLeNet Model

- **1x1 convolution:** This path applies a 1x1 convolution to the input. This reduces the number of channels in the input, which helps to reduce the computational complexity of the network.
- **3x3 convolution:** This path applies a 3x3 convolution to the input. This is a standard convolutional operation that is used to extract features from the input image.
- **5x5 convolution:** This path applies a 5x5 convolution to the input. This path is used to capture larger-scale features from the input image.
- **Max pooling:** This path applies a max pooling operation to the input. This operation reduces the size of the input by keeping the maximum value in each 2x2 window.
- The outputs of the four paths are then concatenated together, and a 1x1 convolution is applied to the result. This final 1x1 convolution helps to reduce the number of channels in the output, and also helps to improve the accuracy of the network.

GoogLeNet Model

4. Auxiliary Classifiers

In the GoogLeNet architecture, there are two auxiliary classifiers in the network. They are placed strategically, where the depth of the feature extracted is sufficient to make a meaningful impact, but before the final prediction from the output classifier. The structure of each auxiliary classifier is mentioned below:

- Average pooling layer (5×5 , stride 3)
- 1×1 convolution (128 filters, ReLU)
- Fully connected layer (1024 units, ReLU)
- Dropout layer (dropout rate = 0.7)
- Fully connected softmax layer (1000 classes)
- The auxiliary losses are added to the main loss with a weight of **0.3** to stabilize training.

GoogLeNet Model

5. Model Architecture

GoogLeNet is a **22-layer deep network** (excluding pooling layers) that emphasizes computational efficiency, making it feasible to run even on hardware with limited resources.

GoogLeNet Model

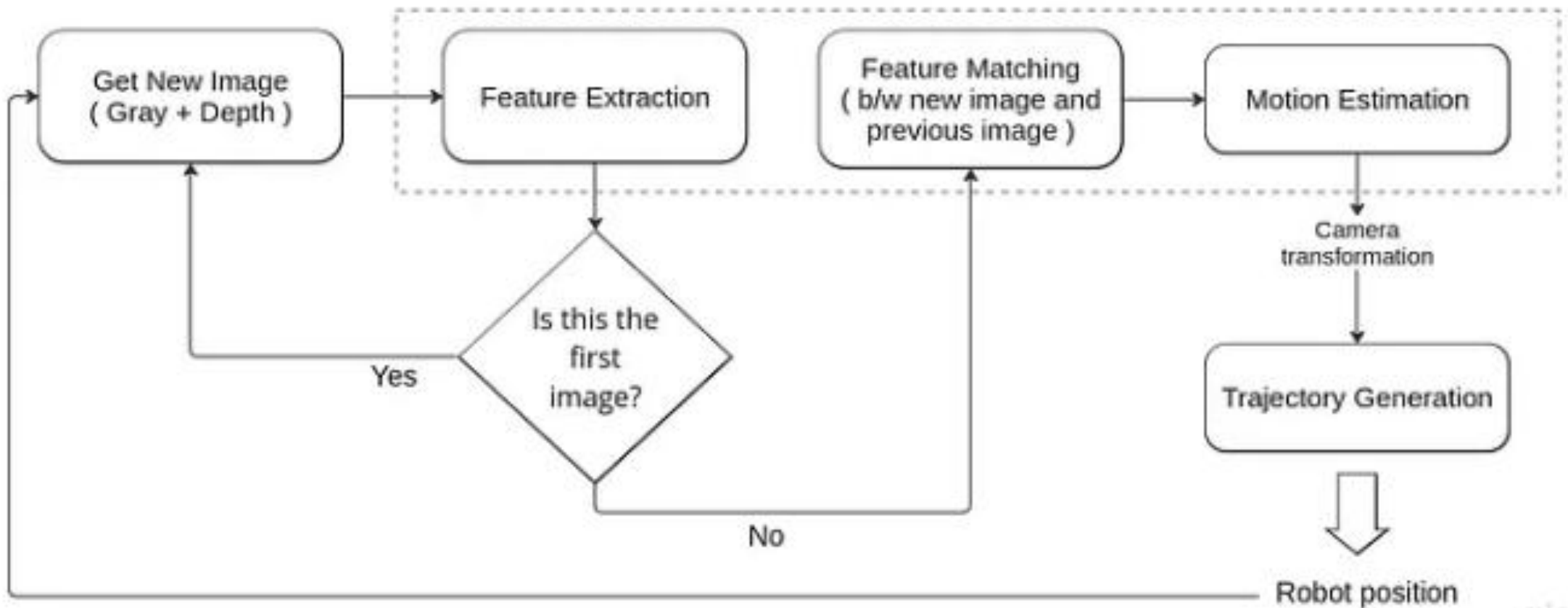
Key highlights of the architecture:

- **Input Layer:** Accepts a 224×224 RGB image as input.
- **Initial Convolutions and Pooling:** Applies a series of standard convolutional and max pooling layers to downsample the input and extract low-level features.
- **Local Response Normalization (LRN):** Normalizes the feature maps early in the network to improve generalization.
- **Inception Modules:** Each module processes the input through 1×1 , 3×3 , and 5×5 convolutions, as well as 3×3 max pooling, all in parallel. The outputs are concatenated along the depth dimension, allowing the network to capture both fine and coarse features.
- **Auxiliary Classifiers:** Appear as smaller branches connected to intermediate layers of the network. Include average pooling, 1×1 convolutions, fully connected layers, and softmax outputs.
- **Final Layers:** Uses global average pooling (7×7) to reduce each feature map to a single value. Followed by a fully connected layer and a softmax activation to produce the final classification output.

Visual Odometry: Frame-to-frame motion estimation

- Visual Odometry (VO) is the process of estimating the egomotion (3D pose: position and orientation) of an agent—such as a robot or vehicle—by analyzing the sequential images captured by an on-board camera.
- It works by tracking the movement of features in the scene across consecutive frames and computing the relative camera transformation between them.
- Unlike Simultaneous Localization and Mapping (SLAM), VO focuses primarily on local, frame-to-frame consistency rather than building a globally optimized map

Visual Odometry: Frame-to-frame motion estimation



Visual Odometry: Frame-to-frame motion estimation

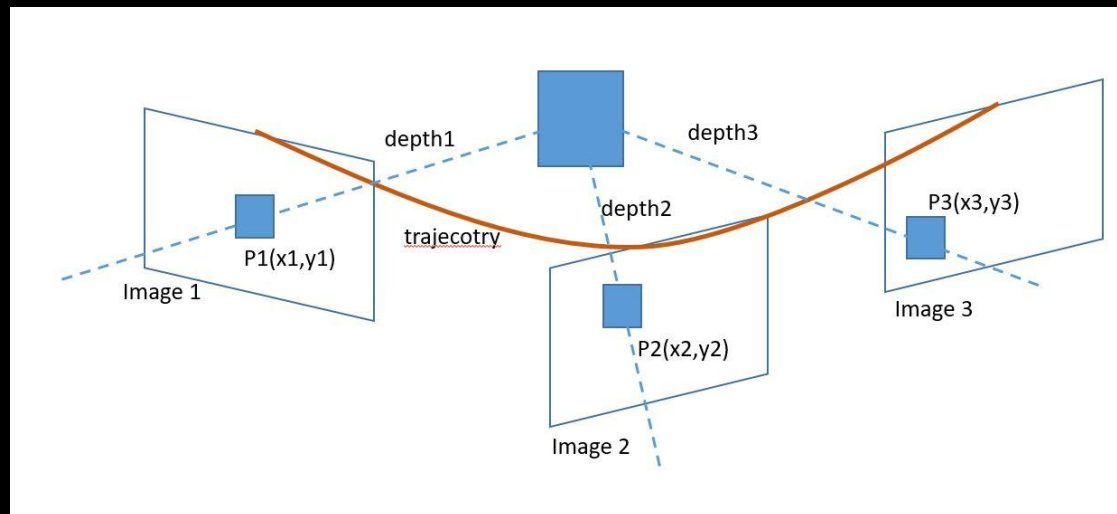
Key Components of Frame-to-Frame VO

- Image Acquisition & Preprocessing: Capturing video frames and applying corrections (e.g., distortion removal).
- Feature Detection & Tracking: Identifying key points (e.g., ORB, FAST, SIFT) in one frame and matching or tracking them in the next to determine their movement (e.g., KLT tracker).
- Motion Estimation: Utilizing geometric constraints (Epipolar geometry, Essential Matrix, PnP) to calculate the rotation (R) and translation (t) that best describes the motion between the two images.
- Local Optimization: Refining the computed camera pose to minimize tracking errors, often using bundle adjustment.

Visual Odometry: Frame-to-frame motion estimation

Types of Visual Odometry

- **Monocular VO:** Uses a single camera. It cannot recover absolute scale, meaning it can only determine motion up to a scale factor.
- **Stereo VO:** Uses two cameras, enabling triangulation to compute depth and motion in metric units.
- **RGB-D VO:** Uses RGB-D cameras (like Kinect) to gain direct 3D structure information



Visual Odometry: Frame-to-frame motion estimation

Advantages and Challenges

- **Advantages:** Lower cost than LIDAR, high-precision in textured environments, and not affected by wheel slip.
- **Challenges:** Susceptible to poor lighting conditions, textureless environments, and high-speed motion blur.
- **Drift Accumulation:** As a frame-to-frame technique, errors accumulate over time, leading to trajectory drift, which is typically addressed by incorporating Loop Closure or using VIO (Visual-Inertial Odometry)

Visual Odometry: Frame-to-frame motion estimation

Performance and Applications

- Modern VO systems, such as ORB-SLAM and feature-based methods, are designed for real-time operation.
- Research in this field focuses on maximizing efficiency and robustness, such as developing lightweight "motion-centric" frameworks that can operate on low-power CPU devices.
- It is heavily used in automotive navigation, UAVs, and robotic mapping.

SLAM (Simultaneous Localization and Mapping)

- SLAM (simultaneous localization and mapping) is a method used for autonomous vehicles that allows user to build a map and localize your vehicle in that map at the same time.
- SLAM algorithms allow the vehicle to map out unknown environments.
- Engineers use the map information to carry out tasks such as path planning and obstacle avoidance.
- SLAM algorithms are useful in many other applications such as navigating a fleet of mobile robots to arrange shelves in a warehouse, parking a self-driving car in an empty spot, or delivering a package by navigating a drone in an unknown environment.

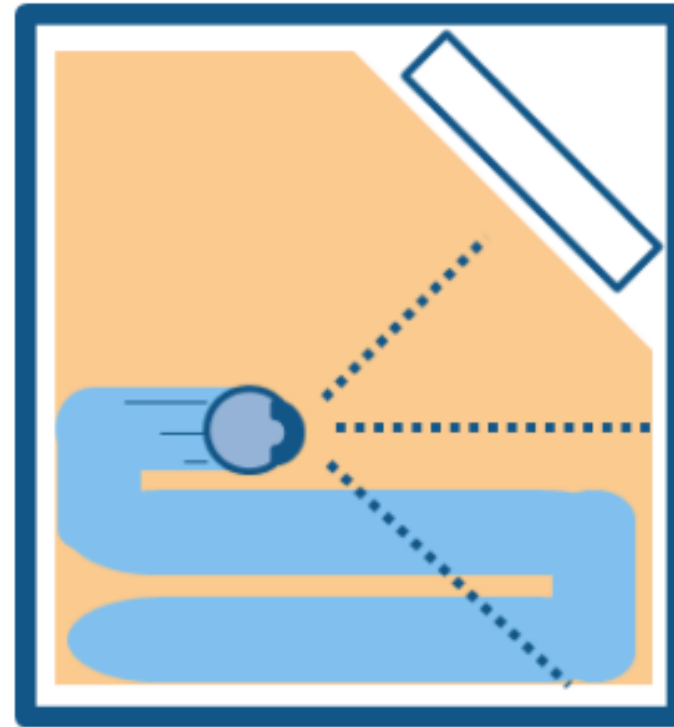
SLAM (Simultaneous Localization and Mapping)

- Consider a home robot vacuum. Without SLAM, it will just move randomly within a room and may not be able to clean the entire floor surface.
- In addition, this approach uses excessive power, so the battery will run out more quickly.
- On the other hand, robots with a SLAM algorithm can use information such as the number of wheel revolutions and data from cameras and other imaging sensors to determine the amount of movement needed.
- This is called localization.
- The robot can also simultaneously use the camera and other sensors to create a map of the obstacles in its surroundings and avoid cleaning the same area twice. This is called mapping.

SLAM (Simultaneous Localization and Mapping)

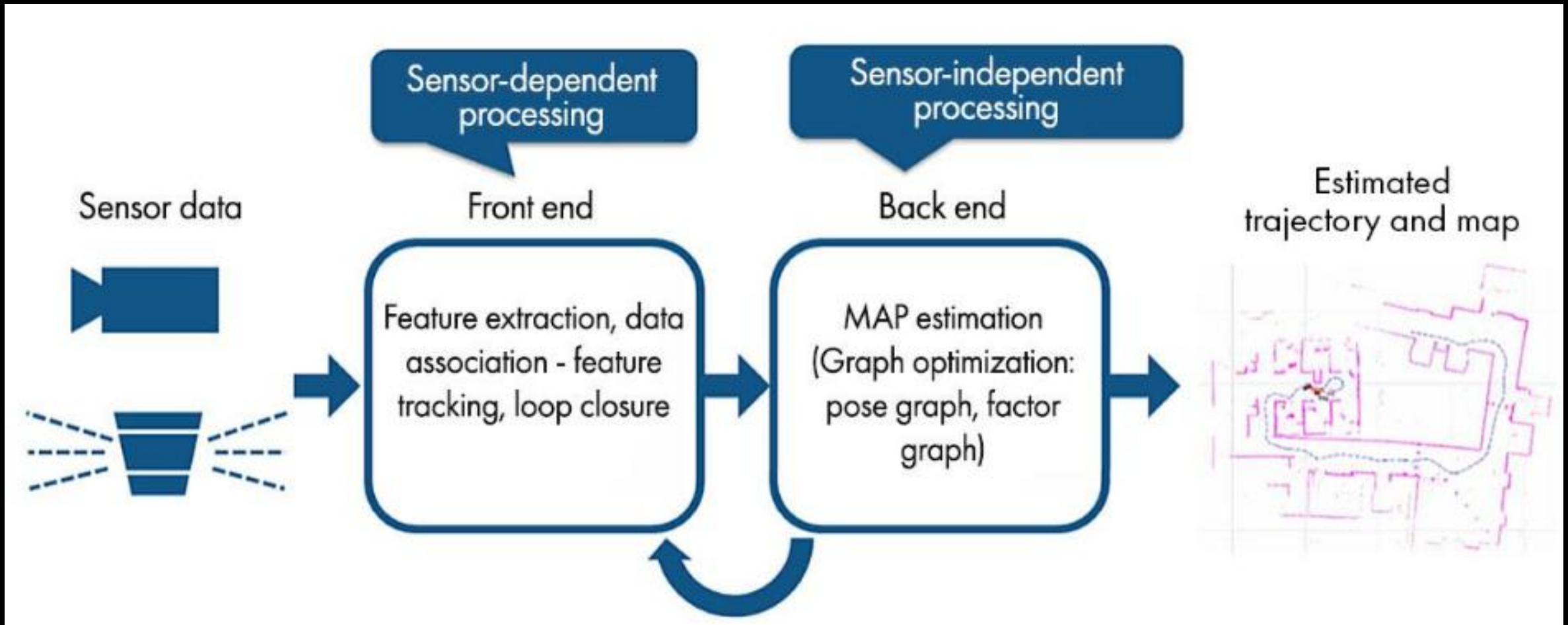


Without SLAM:
Cleaning a room randomly.



With SLAM:
Cleaning while understanding the room's layout.

SLAM (Simultaneous Localization and Mapping)



SLAM (Simultaneous Localization and Mapping)

- There are two types of technology components used to achieve SLAM.
- The first type is sensor signal processing, including the front-end processing, which is largely dependent on the sensors used.
- The second type is pose-graph optimization, including the back-end processing, which is sensor-agnostic.

SLAM (Simultaneous Localization and Mapping) - Types of SLAM Methods

Visual SLAM

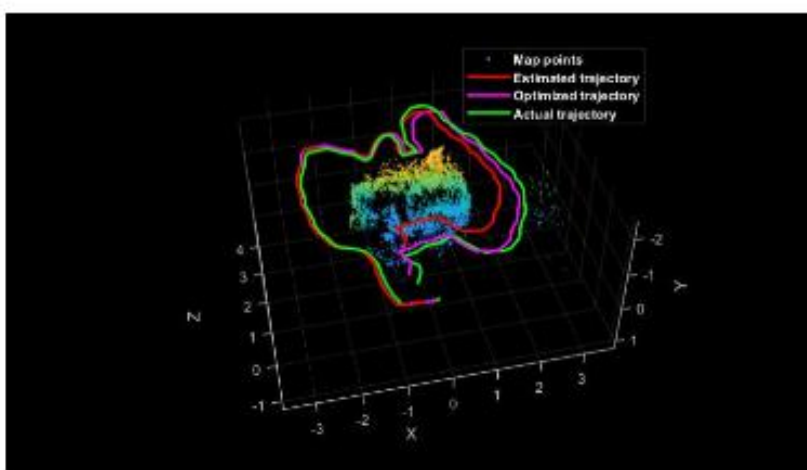
- Visual SLAM (vSLAM) uses camera-based images to simultaneously map an environment and estimate position.
- It works with various camera types, including monocular, stereo, and RGB-D cameras, making it a cost-effective solution.
- Cameras provide rich visual data, enabling landmark detection and integration with graph-based optimization for flexible implementation.

SLAM (Simultaneous Localization and Mapping) - Types of SLAM Methods

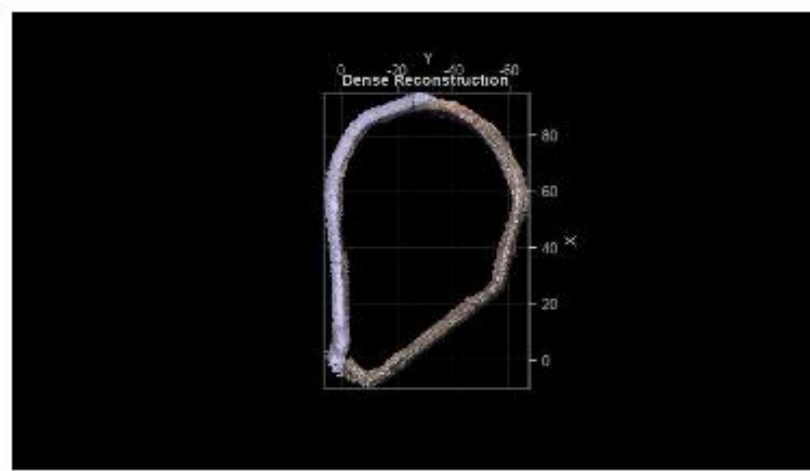
Visual SLAM

- Monocular SLAM, which uses a single camera, faces challenges in estimating depth.
- This can be addressed using known reference objects (like markers or checkerboards) or by combining data with sensors such as IMUs.
- Related techniques include structure from motion, visual odometry, and bundle adjustment.
- vSLAM methods are mainly classified into **sparse methods**, which track feature points (e.g., PTAM, ORB-SLAM), and **dense methods**, which use full image intensity information (e.g., DTAM, LSD-SLAM, DSO, SVO).

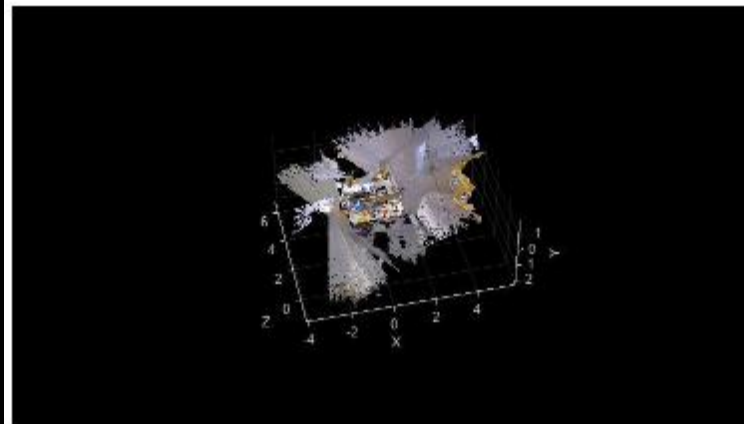
SLAM (Simultaneous Localization and Mapping) - Visual SLAM



Monocular vSLAM



Stereo vSLAM

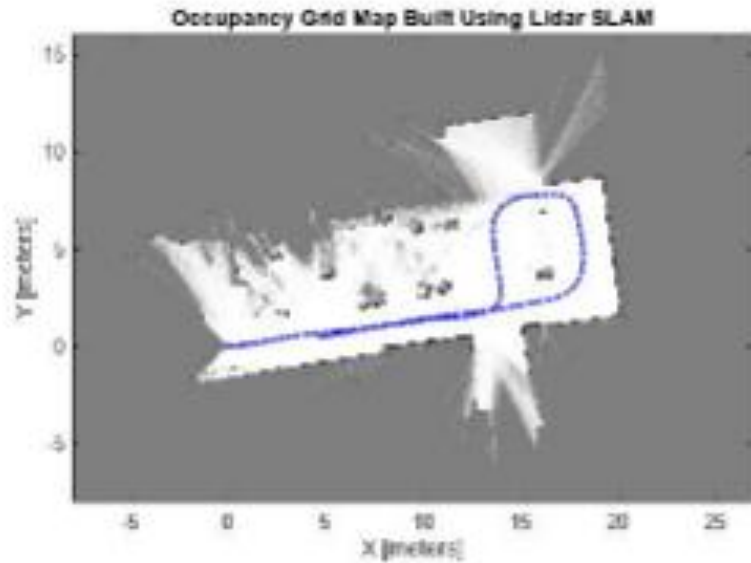


RGB-D vSLAM

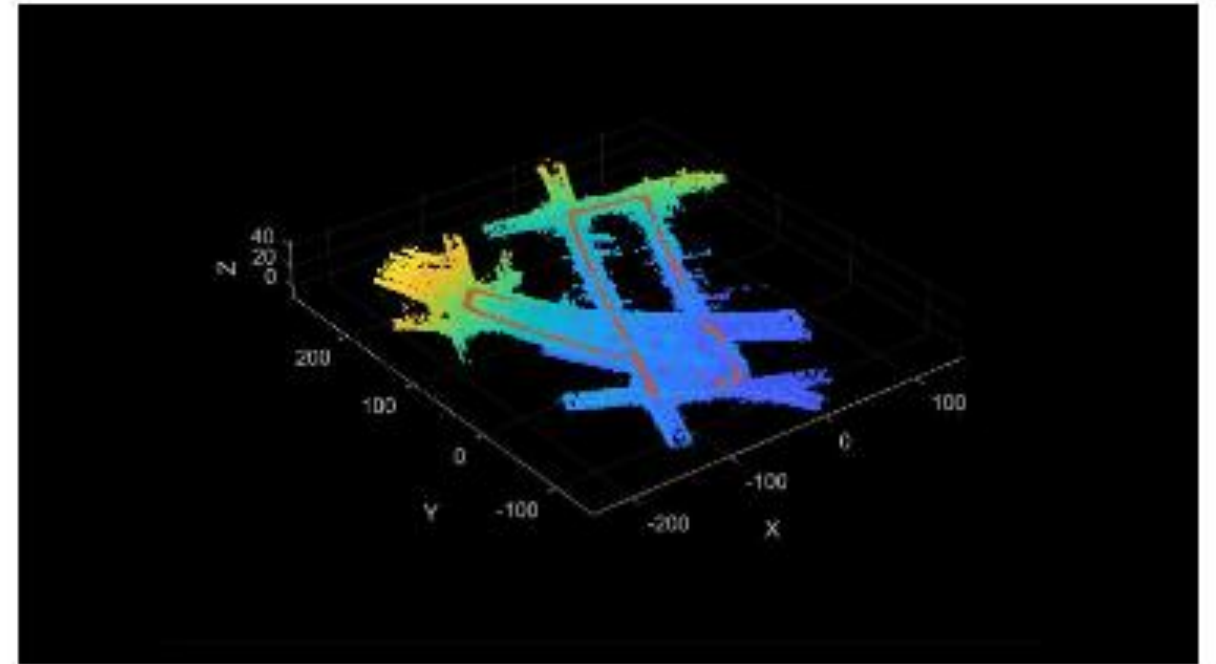
SLAM (Simultaneous Localization and Mapping) - Lidar SLAM

- LiDAR (Light Detection and Ranging) uses laser sensors to measure distances with high precision, making it suitable for applications like self-driving cars and drones.
- It generates 2D or 3D point cloud data, which is used in SLAM for accurate map building and localization by sequentially aligning scans.
- Movement estimation is performed using registration algorithms such as ICP and NDT, or feature-based methods like LOAM and FGR. Point cloud maps can be represented as grid or voxel maps.
- To improve localization accuracy, LiDAR is often combined with other sensors like wheel odometry, GNSS, and IMUs.
- Typically, 2D LiDAR SLAM is used in indoor environments (e.g., warehouse robots), while 3D LiDAR SLAM is preferred for UAVs and autonomous vehicles.

SLAM (Simultaneous Localization and Mapping) - Lidar SLAM



SLAM with 2D LiDAR



SLAM with 3D LiDAR

SLAM (Simultaneous Localization and Mapping) - Multi-Sensor SLAM

- Multi-sensor SLAM combines data from multiple sensors—such as cameras, IMUs, GPS, lidar, and radar—to improve accuracy and reliability.
- It leverages the strengths of each sensor while compensating for their individual weaknesses (e.g., cameras struggle in low light, while lidar may have issues with certain surfaces).
- By integrating diverse sensor inputs, multi-sensor SLAM provides more robust performance than single-sensor systems.
- A commonly used framework is the factor graph, which flexibly fuses data from different sensors by representing them as pose factors.
- This allows the development of systems like visual-inertial SLAM and lidar-IMU SLAM.

Common Challenges with SLAM

1. Localization errors accumulate, causing substantial deviation from actual values

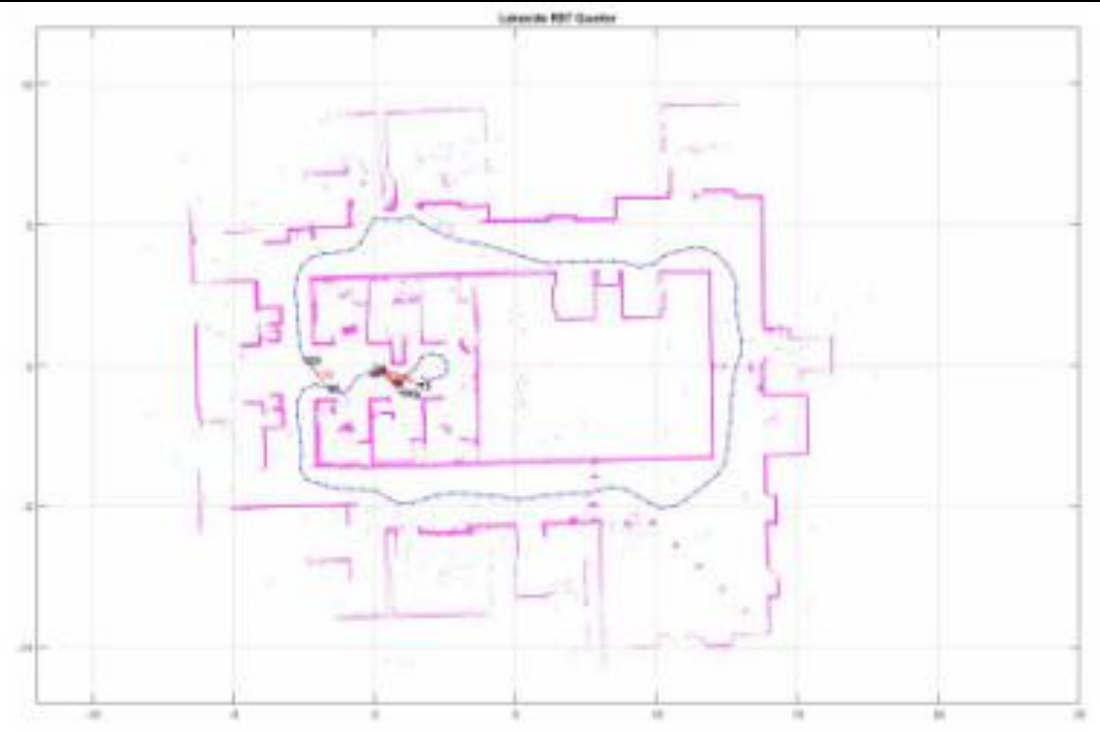
- SLAM algorithms accumulate small errors over time, leading to drift in position and map distortion.
- This becomes evident in scenarios like a robot moving in a loop, where the starting and ending positions do not align—known as the loop closure problem.
- Detecting loop closures is essential to correct these accumulated errors.
- In multi-sensor SLAM, accurate sensor calibration is crucial, as misalignment can cause fusion errors and degrade performance.
- Techniques like factor graph optimization help improve calibration, especially for systems combining cameras and IMUs.
- To reduce localization errors, SLAM systems store features of previously visited places as landmarks.
- Pose graphs are used to model relationships between positions, and optimization techniques are applied to minimize errors. In visual SLAM, this optimization process is called bundle adjustment, which enhances both map accuracy and localization.

Common Challenges with SLAM

1. Localization errors accumulate, causing substantial deviation from actual values



Example of constructing a pose graph and minimizing errors.



Example of constructing a pose graph and minimizing errors.

Common Challenges with SLAM

2. Localization fails and the position on the map is lost

- Image- and point-cloud-based mapping alone may ignore a robot's motion constraints, leading to unrealistic position estimates (e.g., sudden large jumps). To prevent such localization failures, SLAM systems use recovery algorithms or combine motion models with sensor data.
- A common approach is sensor fusion with motion models using filtering techniques like Kalman filters. Since many robots have nonlinear motion, methods such as Extended Kalman Filters (EKF), particle filters (Monte Carlo localization), and Unscented Kalman Filters (UKF) are used.
- Sensors like IMUs, AHRS, INS, gyroscopes, accelerometers, magnetometers, and wheel encoders support accurate localization.
- For recovery, SLAM systems store landmarks as keyframes from previously visited locations. Feature extraction methods like Bag of Features (BoF) and Bag of Visual Words (BoVW), along with newer deep learning techniques, help quickly recognize these landmarks and restore accurate positioning.

Common Challenges with SLAM

3. High computational cost for image processing, point cloud processing, and optimization

- SLAM algorithms are computationally intensive, which makes implementation on low-power embedded systems challenging.
- Tasks like image processing, point cloud matching, and optimization processes (e.g., loop closure) require high-frequency execution for accurate localization.
- To address this, parallel processing is used. Operations like feature extraction can be efficiently parallelized using multicore CPUs, SIMD techniques, and embedded GPUs to improve speed.
- Additionally, computationally heavy tasks such as pose graph optimization can be executed less frequently or with lower priority, helping balance performance and resource usage on embedded hardware.

ArUco markers

- **ArUco markers are square fiducial markers** used in computer vision to estimate **position and orientation (pose)** of a camera or object.
They consist of:
 - A **black square border**
 - An **inner binary pattern (grid of black & white cells)**
 - A **unique ID** encoded in the pattern
- They are widely used in **robotics, SLAM, augmented reality, and camera calibration.**

ArUco markers

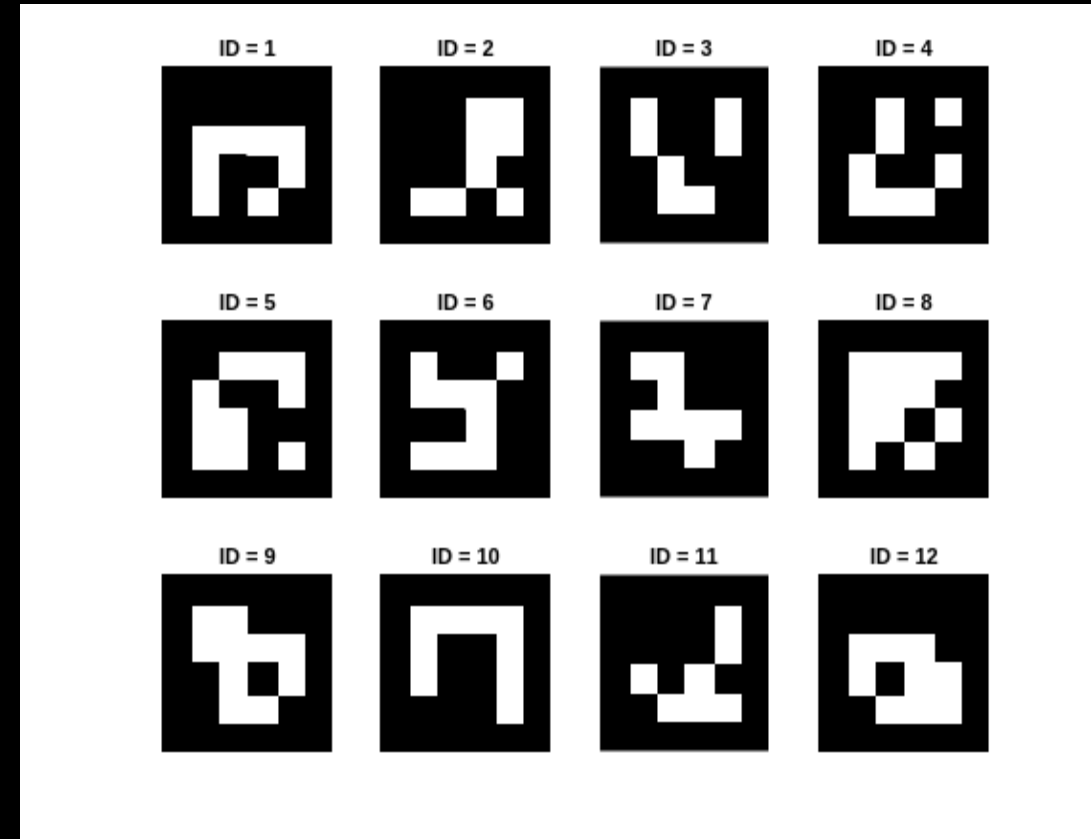
Structure of an ArUco Marker

An ArUco marker typically has:

- Outer black border → helps in detection
- Inner grid (e.g., 4×4 , 5×5 , 6×6) → stores binary information
- Each grid cell = 0 (white) or 1 (black)

Example:

A **5×5 marker** has 25 bits encoding its ID
Predefined dictionaries ensure **no two markers are easily confused**



ArUco markers

- Markers are grouped into **dictionaries**, such as:
 - DICT_4X4_50 → 50 markers, each 4×4 grid
 - DICT_5X5_100 → 100 markers, each 5×5 grid
 - DICT_6X6_250 → larger and more robust

Larger grids:

- More unique IDs
- Better error detection
- Slightly slower detection

ArUco markers

Step-by-step detection:

1. **Capture Image** from camera
2. **Convert to grayscale**
3. **Thresholding** → detect black/white regions
4. **Find square contours**
5. **Filter candidates** based on shape
6. **Perspective correction** (warp square to flat view)
7. **Decode binary pattern** → identify marker ID
8. **Estimate pose** using camera calibration

ArUco markers - Pose Estimation

Once detected, we compute:

- **Translation (x, y, z)** → position
- **Rotation (roll, pitch, yaw)** → orientation

This uses:

- Known **marker size**
- Camera parameters (intrinsic matrix)

Output:

- Rotation vector (**rvec**)
- Translation vector (**tvec**)

These are used in:

- Robot localization
- Object tracking
- AR overlays

ArUco markers

Role in SLAM

- ArUco markers act as **artificial landmarks**:
- Help solve **monocular depth ambiguity**
- Improve **localization accuracy**
- Assist in **loop closure detection**
- Reduce **drift over time**

Applications

- **Robotics navigation**
- **SLAM systems**
- **Augmented Reality (AR)**
- **Camera calibration**
- **Drone landing systems**
- **Industrial automation**

ArUco markers

Advantages

- Low-cost (just printed markers)
- Easy to detect
- Unique identification
- Works in real-time
- Robust to rotation and partial occlusion

Limitations

- Sensitive to lighting conditions
- Requires clear visibility
- Limited range (depends on camera resolution)
- Artificial markers (not natural features)